

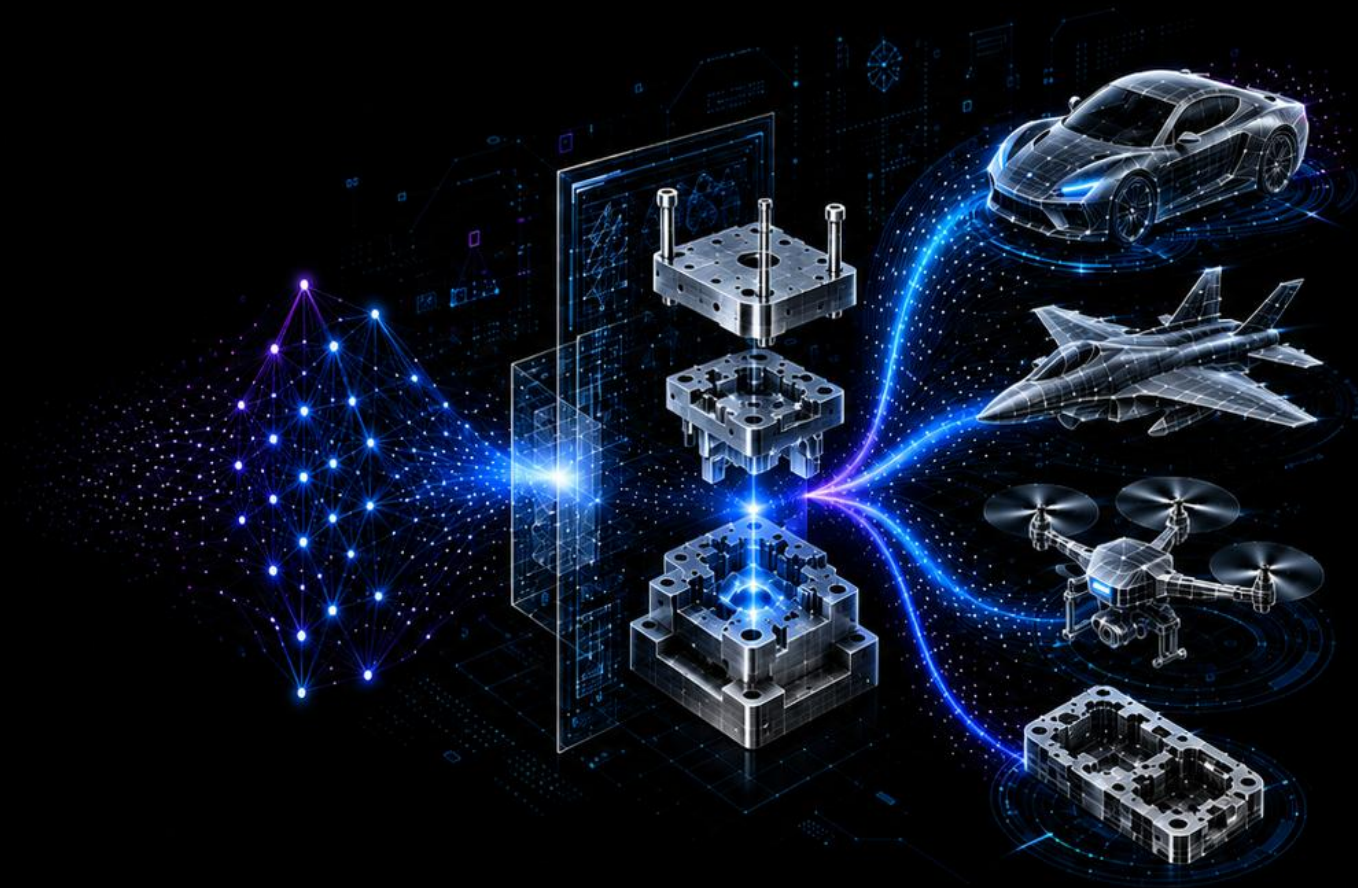
개인 과제 발표

Text2CAD - 2

최준석

(지도교수: 양선웅)

한양대학교 ERICA 기계공학과



Intelligent Design
for Engineering
Applications Lab

■ 초기 구성

LLM을 통해서 Free CAD를 만들기 위해 초기 세팅을 다음과 같이 설정함.

Local LLM을 사용하고 싶어서 추천 모델인 Ollama 사용

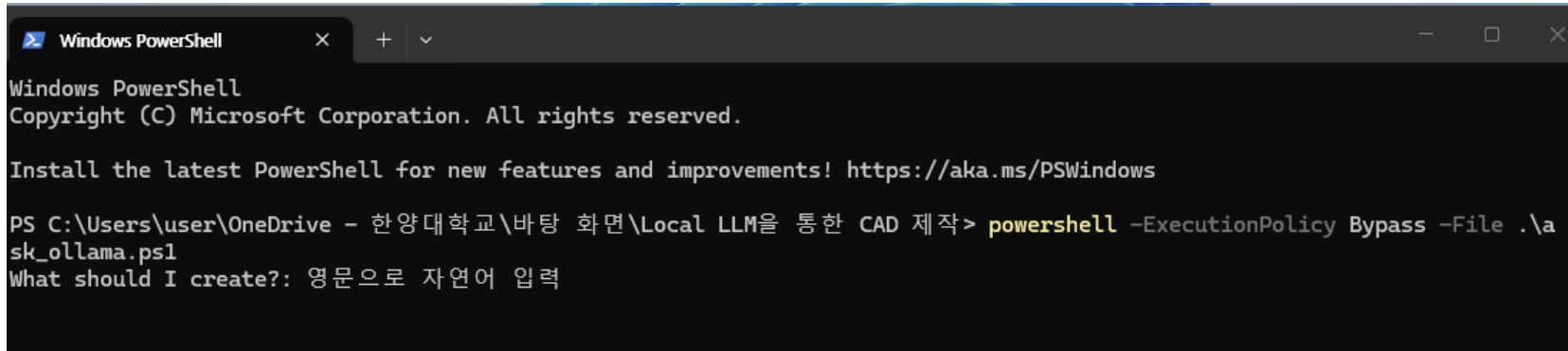
Free CAD는 웹사이트 상 최신 버전

Qwen2.5-Coder 7B는 FreeCAD Python 코드 생성해주는 기능

```
graph TD; A[사용자 입력] --> B[try_it.bat 클릭 (프롬프트)]; B --> C[PowerShell 실행]; C --> D[Ollama 로컬 서버]; D --> E[Qwen2.5-Coder 7B]; E --> F[FreeCAD Python 코드 생성]; F --> G[PowerShell 안전 정제]; G --> H[FreeCADCmd 실행]; H --> I[형상 유효성 검사]; I --> J[result.FCStd 저장];
```

STEP 1을 위한 초기 설계 모델 (어떤 방식으로 실행이 되는가?)

1. 파일에 들어가서 우클릭 후 터미널 실행
 2. Powershell -ExecutionPolicy Bypass -File .\ask_ollama.ps1 코드 입력
 3. What should I create?: 에 영문으로 만들고 싶은 형상에 대해 자세히 기입
 4. 자연어 입력 후 generated.py 파일 만들어 지고 실행 시켜 치수가 잘 입력 되었는지 확인
 5. 다시 파일의 터미널 실행
 6. powershell -ExecutionPolicy Bypass -File .\run_freecad.ps1 코드 입력
 7. 자동으로 Free CAD가 켜지고 육안으로 생성된 파일 확인
- + 더 자세한 치수가 보고 싶을 땐 보기 -> 파이썬 명령창으로 자세한 치수 확인 가능



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\user\OneDrive - 한양대학교\바탕 화면\Local LLM을 통한 CAD 제작> powershell -ExecutionPolicy Bypass -File .\ask_ollama.ps1
What should I create?: 영문으로 자연어 입력
```

1. What should I create? 출력
2. 사용자의 영어 요구사항 입력
3. Ollama의 로컬 API 호출
4. qwen2.5-coder:7b 실행
5. Qwen이 FreeCAD Python 코드 생성
6. Markdown 표시 제거
7. generated.py로 저장

generated.py 확인

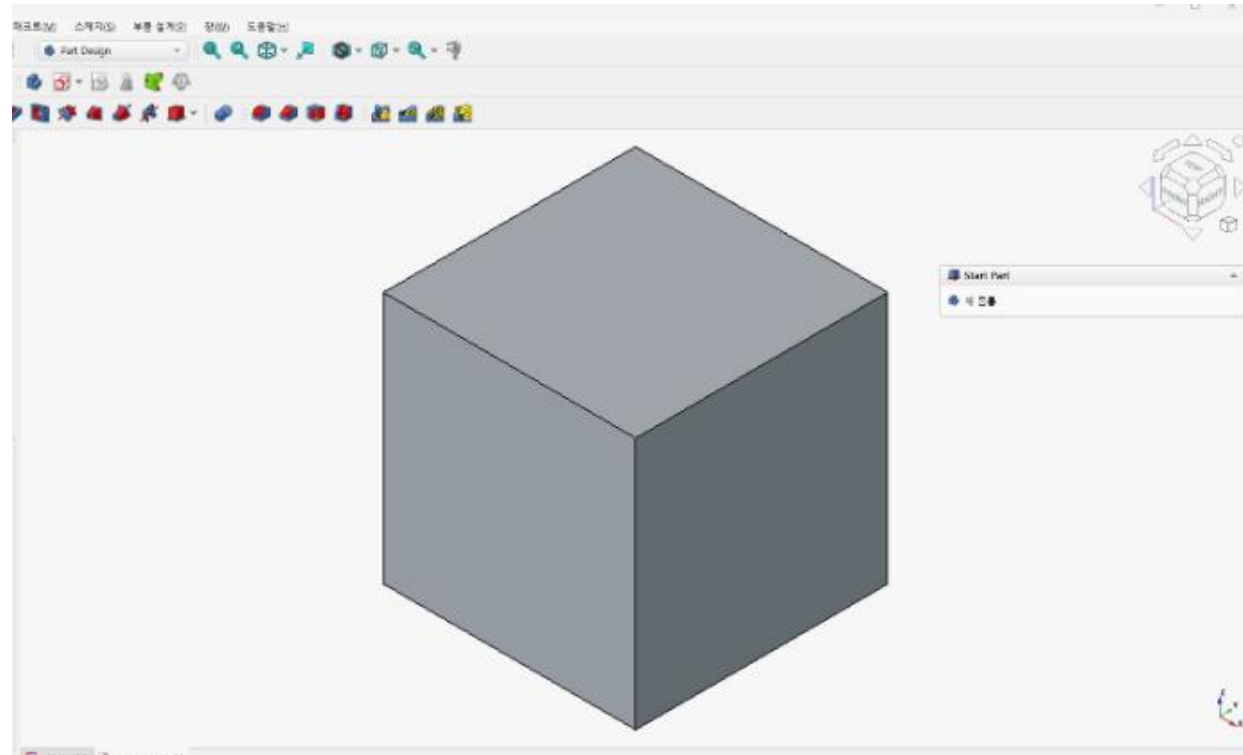
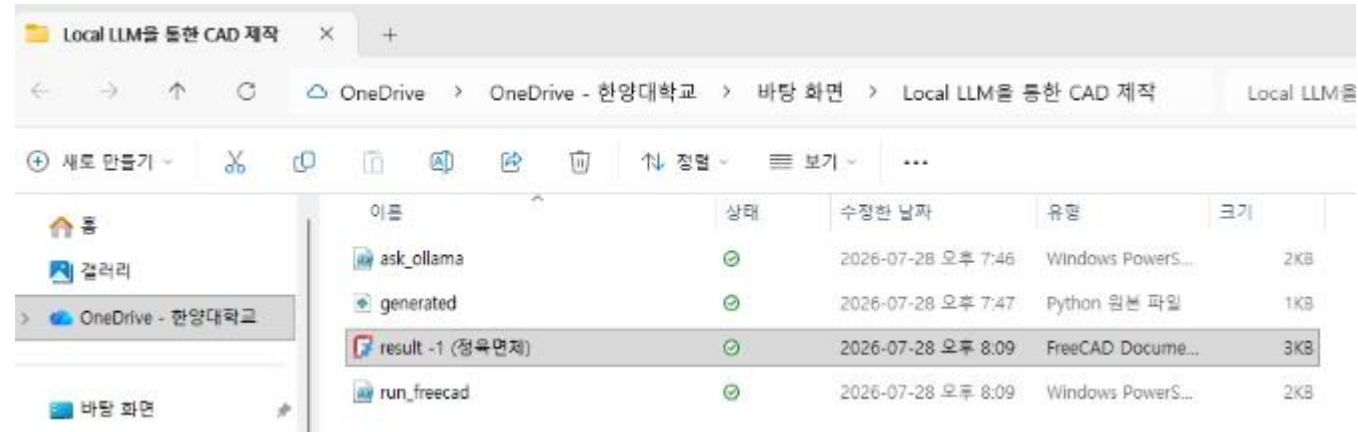
- 영문 임시 폴더로 복사
- FreeCADCmd 실행
- generated.py를 FreeCAD에서 실행
- result.FCStd 저장
- 결과 파일을 프로젝트 폴더로 복사
- FreeCAD GUI로 결과 열기

STEP 1: 정육면체

영어 자연어 요청

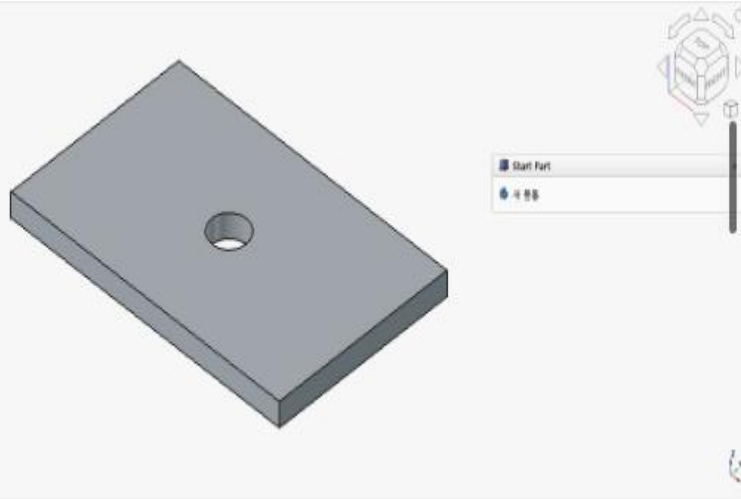
- 로컬 LLM 호출
- FreeCAD Python 코드 생성
- 코드 파일 저장
- FreeCAD에서 실행
- FCStd 파일 생성
- 결과 자동 열기

=> 성공적으로 정육면체 생성



STEP 2 : 중앙 구멍이 있는 80 × 50 × 5mm 판이 나오는 실험

요청한 자연어값: Create a rectangular plate with X length 80 mm, Y width 50 mm, and Z thickness 5 mm. Subtract one cylindrical through hole along the positive Z direction. The hole diameter is 10 mm and its center is at X=40 mm, Y=25 mm. Store the final shape in one Part::Feature named Result.

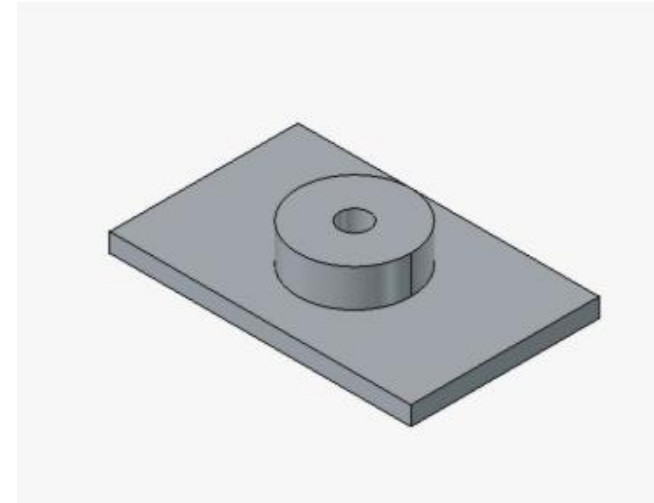


```
>>> # Gui.Selection.addSelection('result','Result','',43.7438,28.3138,5)
>>> # Gui.Selection.clearSelection()
>>> # Gui.Selection.addSelection('result','Result','Edge10',43.7438,28.3138,5)
>>> # Gui.Selection.clearSelection()
>>> # Gui.Selection.addSelection('result','Result','',44.4371,27.2978,5)
>>> obj = App.ActiveDocument.getObject("Result")
>>> print(obj.Shape.BoundingBox.XLength)
80.0
>>> print(obj.Shape.BoundingBox.YLength)
50.0
>>> print(obj.Shape.BoundingBox.ZLength)
5.0
```

⇒ 성공적으로 생성

STEP 3 : 80×50×5 mm 베이스 판 + 중앙에 반지름 15mm, 높이 10mm 원통형 보스 - 중앙에 지름 8mm 관통구멍

요청한 자연어값: Create one solid mechanical part in FreeCAD. Create a rectangular base plate with X length 80 mm, Y width 50 mm, and Z thickness 5 mm. Create a cylindrical boss with radius 15 mm and height 10 mm. Center the boss at X=40 mm and Y=25 mm. The boss must start on the top surface of the plate at Z=5 mm and extend in the positive Z direction. Fuse the boss and the base plate into one solid. Create a cylindrical through hole with diameter 8 mm. Center the hole at X=40 mm and Y=25 mm. The hole must start at Z=0 and extend 15 mm in the positive Z direction. Subtract the hole from the fused solid. Store the final shape in one Part::Feature named Result.

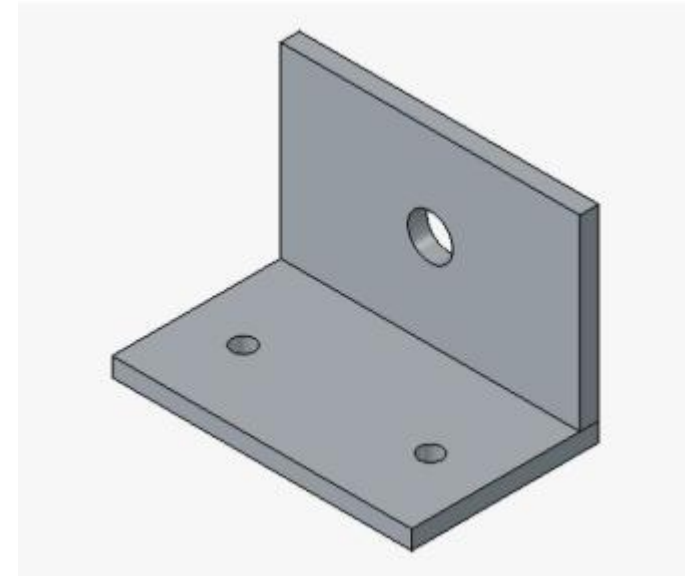


```
>>> ### Begin command Std_ViewFitAll
>>> # Gui.SendMsgToActiveView("ViewFit")
>>> ### End command Std_ViewFitAll
>>> obj = App.ActiveDocument.getObject("Result")
>>> print(obj.Shape.BoundingBox.XLength, obj.Shape.BoundingBox.YLength, obj.Shape.BoundingBox.ZLength)
80.0 50.0 15.0
>>> print(len(obj.Shape.Solids))
1
>>> print(obj.Shape.Volume)
26314.60123371549
>>> |
```

⇒ 여기까지의 단점은 무조건 영문으로 입력하고 띄어쓰기 없이 입력 해야 한다.
그래도 성공적으로 생성

STEP 4 : L자형 장착 브래킷

요청한 자연어값: Create one L-shaped mounting bracket as a single solid in FreeCAD. Create a horizontal base plate with X length 80 mm, Y width 50 mm, and Z thickness 5 mm. Its origin is X=0, Y=0, Z=0. Create a vertical wall using a box with X length 80 mm, Y thickness 5 mm, and Z height 50 mm. Place the wall at X=0, Y=45, Z=5. Fuse the wall with the base plate. Create two cylindrical through holes in the horizontal base. Each hole has diameter 6 mm, starts at Z=0, has height 5 mm, and points along positive Z. Place their centers at X=15, Y=20 and X=65, Y=20. Create one cylindrical through hole in the vertical wall. The hole has diameter 12 mm. Its axis points along positive Y. Start it at X=40, Y=45, Z=30 and give it a length of 5



```
>>> obj = App.ActiveDocument.getObject("Result")
>>> [(print("radius=", e.Curve.Radius, "center=", e.Curve.Center, "axis=", e.Curve.Axis)) for e in obj.Shape.Edges if hasattr(e.Curve, "Radius")]
radius= 3.0 center= Vector (15.0, 20.0, 5.0) axis= Vector (0.0, 0.0, 1.0)
radius= 3.0 center= Vector (65.0, 20.0, 5.0) axis= Vector (0.0, 0.0, 1.0)
radius= 3.0 center= Vector (15.0, 20.0, 0.0) axis= Vector (0.0, 0.0, 1.0)
radius= 3.0 center= Vector (65.0, 20.0, 0.0) axis= Vector (0.0, 0.0, 1.0)
radius= 6.0 center= Vector (40.0, 45.0, 30.0) axis= Vector (0.0, 1.0, 0.0)
radius= 6.0 center= Vector (40.0, 50.0, 30.0) axis= Vector (0.0, 1.0, 0.0)
[None, None, None, None, None, None]
```

⇒ 성공적으로 생성

STEP 5로 넘어가기 전 문제 상황

⇒ 형상이 복잡해지자, generated.py 파일에 치수 오류가 발생함.

그래서 1차 생성 후, 로컬 모델이 원래 요구사항과 코드를 비교해 지름·반지름·축 방향·좌표를 자동 교정하는 2차 검토 단계를 추가적으로 넣어 줌.

→ ask_ollama.ps1을 다음 구조로 확장

따라서, 로컬 LLM이 FreeCAD Python 코드를 직접 작성하지 않고, 먼저 구조화된 CAD 설계 명세를 만들게 하려고 새로운 파일(ask_plan)을 만들어줌.

why: 어느 방식이 더 정확하고 복잡한 형상을 안정적으로 만드는지 비교

※ 기존 방식 ※

자연어 → LLM이 Python 전체 생성

[사용자 요청

→ Qwen이 generated.py 직접 작성

→ FreeCAD 실행]

※ 개선 방식 ※

자연어 → LLM이 JSON 설계 명세 생성

→ 우리가 만든 변환기가 정확한 Python 생성

→ FreeCAD 실행·검증

[ask_plan.ps1

자연어 → plan.json

다음에 만들 변환기

plan.json → generated.py

run_freecad.ps1

generated.py → result.FCStd]

■ 문제상황 두 번째

⇒ STEP 5를 다시 실행했을 때, 바꾼 모델도 수치적인 부분에 누락이 존재..

새로운 해결 구조

(ask_plan 파일을 요구 사항 목록과 CAD 계획을 따로 생성 하고,
누락 시 최대 3회를 자동 수정하는 방식으로 교체)

사용자 요청



1. 요구사항 추출기
필요한 박스·구멍 목록 작성



2. CAD 계획 생성기
plan.json 작성



3. PowerShell 비교
요구사항과 plan.json 비교



일치 → FreeCAD 코드 생성
불일치 → 누락 정보를 Qwen에 전달해 수정

[새로운 해결 구조 설명]

1. Qwen 호출은 CAD 계획을 만들지 않고 요구사항만 추출한다.
2. 호출은 실제 치수와 좌표가 포함된 plan.json을 만듦.
3. PowerShell이 개수를 비교.
4. 실패하면 자동으로 다음 피드백을 보냄 .
5. 수정된 JSON이 다음 조건을 모두 통과해야만 FreeCAD 단계로 넘어감. (최대 3회 수정 후 넘어가면, 정지)

⇒ 정상 작동 확인. 실제 체크 리스트 길이를 기준으로 박스 3개·구멍 6개 를 검증했고, 치수·좌표·방향도 모두 정확한 plan.json 이 만들어짐.

→ 비교 후에 JSON에 Free CAD Python 코드로 변환하는 컴파일러 추가

■ 업데이트 된 과정 추가 설명

최종 과정 (3가지 명령 단계)

```
# 1. 자연어를 검증된 CAD 계획으로 변환
powershell -ExecutionPolicy Bypass -File .\ask_plan.ps1

# 2. 계획을 안전한 FreeCAD 코드로 변환
powershell -ExecutionPolicy Bypass -File .\compile_plan.ps1

# 3. FreeCAD 실행.검증.결과 열기
powershell -ExecutionPolicy Bypass -File .\run_freecad.ps1
```

새로운 방식

자연어
→ Qwen
→ plan.json
→ compile_plan.ps1
→ generated.py
→ FreeCAD

Qwen은 다음처럼 설계 정보만 작성합니다.

```
{
  "diameter": 16,
  "length": 10,
  "position": [0, 30, 35],
  "direction": [1, 0, 0]
}
```

`plan.json`은 **LLM이 이해한 CAD 설계 내용을 저장하는 중간 설계서**

`compile\_plan.ps1`은 설계서를 바탕으로 계산해주는 코드.

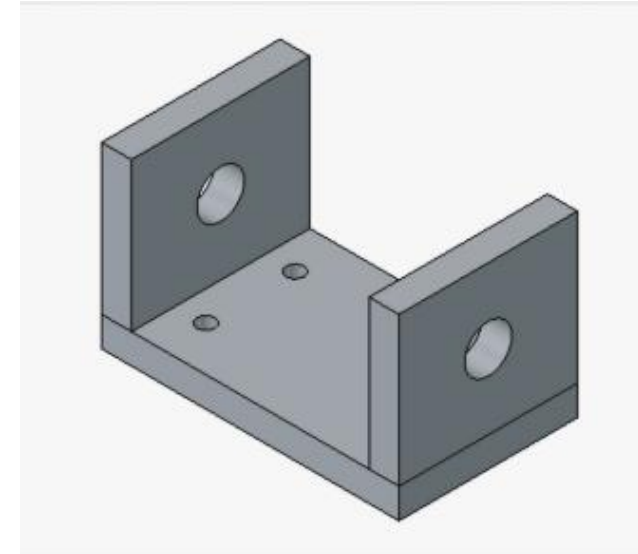
STEP 5 : U자형 샤프트 지지 브래킷

요청한 자연어값: Create one U-shaped shaft support bracket as a single solid in FreeCAD. Create a horizontal base plate with X length 100 mm, Y width 60 mm, and Z thickness 10 mm at origin X=0, Y=0, Z=0. Create a left vertical wall using a box with X thickness 10 mm, Y width 60 mm, and Z height 50 mm at X=0, Y=0, Z=10. Create a right vertical wall using a box with X thickness 10 mm, Y width 60 mm, and Z height 50 mm at X=90, Y=0, Z=10. Fuse the base plate and both walls into one solid. Create four cylindrical mounting holes through the base plate. Each mounting hole has diameter 6 mm, starts at Z=0, has height 10 mm, and points along positive Z. Place the hole centers at X=20,Y=15; X=80,Y=15; X=20,Y=45; and X=80,Y=45. Create one cylindrical shaft hole through the left wall with diameter 16 mm. Start it at X=0,Y=30,Z=35, give it length 10 mm, and point it along positive X. Create an identical shaft hole through the right wall. Start it at X=90,Y=30,Z=35, give it length 10 mm, and point it along positive X. Subtract all six holes from the fused bracket. Add only the final shape to the document as one Part::Feature named Result.

```
>>> obj = App.ActiveDocument.getObject("Result")
>>> print(obj.Shape.BoundingBox.XLength, obj.Shape.BoundingBox.YLength, obj.Shape.BoundingBox.ZLength)
100.0 60.0 60.0
>>> print(len(obj.Shape.Solids))
1
>>> print(obj.Shape.Volume)
114847.78804811272
```

```
>>> [(print("diameter=", e.Curve.Radius*2, "center=", e.Curve.Center, "axis=", e.Curve.Axis) for e in obj.Shape.Edges if hasattr(e.Curve, "Radius"))]
diameter= 16.0 center= Vector (0.0, 30.0, 35.0) axis= Vector (1.0, 0.0, -0.0)
diameter= 6.0 center= Vector (20.0, 15.0, 0.0) axis= Vector (0.0, 0.0, 1.0)
diameter= 6.0 center= Vector (80.0, 15.0, 0.0) axis= Vector (0.0, 0.0, 1.0)
diameter= 6.0 center= Vector (20.0, 45.0, 0.0) axis= Vector (0.0, 0.0, 1.0)
diameter= 6.0 center= Vector (80.0, 45.0, 0.0) axis= Vector (0.0, 0.0, 1.0)
diameter= 6.0 center= Vector (20.0, 15.0, 10.0) axis= Vector (0.0, 0.0, 1.0)
diameter= 6.0 center= Vector (80.0, 15.0, 10.0) axis= Vector (0.0, 0.0, 1.0)
diameter= 6.0 center= Vector (20.0, 45.0, 10.0) axis= Vector (0.0, 0.0, 1.0)
diameter= 6.0 center= Vector (80.0, 45.0, 10.0) axis= Vector (0.0, 0.0, 1.0)
```

```
diameter= 6.0 center= Vector (20.0, 45.0, 0.0) axis= Vector (0.0, 0.0, 1.0)
diameter= 6.0 center= Vector (80.0, 45.0, 0.0) axis= Vector (0.0, 0.0, 1.0)
diameter= 6.0 center= Vector (20.0, 15.0, 10.0) axis= Vector (0.0, 0.0, 1.0)
diameter= 6.0 center= Vector (80.0, 15.0, 10.0) axis= Vector (0.0, 0.0, 1.0)
diameter= 6.0 center= Vector (20.0, 45.0, 10.0) axis= Vector (0.0, 0.0, 1.0)
diameter= 6.0 center= Vector (80.0, 45.0, 10.0) axis= Vector (0.0, 0.0, 1.0)
diameter= 16.0 center= Vector (10.0, 30.0, 35.0) axis= Vector (1.0, 0.0, -0.0)
diameter= 16.0 center= Vector (100.0, 30.0, 35.0) axis= Vector (1.0, 0.0, -0.0)
diameter= 16.0 center= Vector (90.0, 30.0, 35.0) axis= Vector (1.0, 0.0, -0.0)
[None, None, None, None, None, None, None, None, None, None]
```



이번 파일은 다음 새 과정으로 만든 결과

자연어

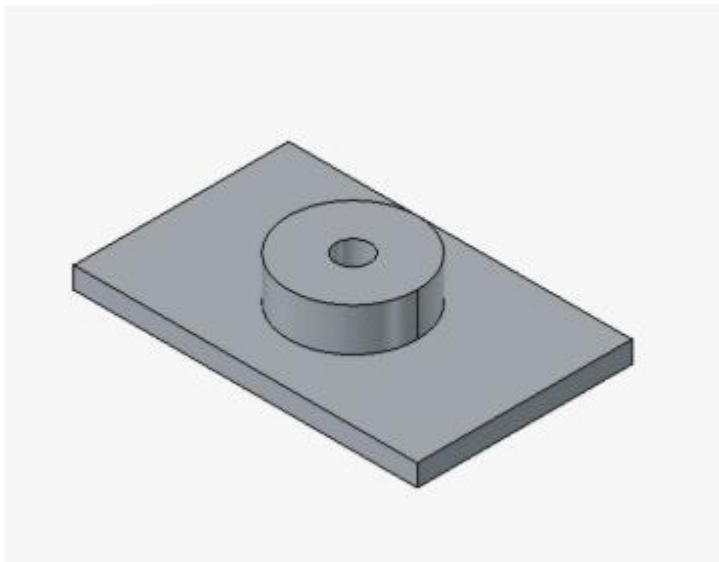
- requirements.json
- 검증된 plan.json
- compile_plan.ps1
- generated.py
- FreeCAD
- result.FCStd

⇒ 결국 자연어를 기반으로
요구 사항을 요청하고 그걸
Plan이 저장 한 뒤, 설계 내용을
잘 이해했는지 확인 후에 맞으면
compile로 계산하여
Generated 파일 생성해서
카티아 실행

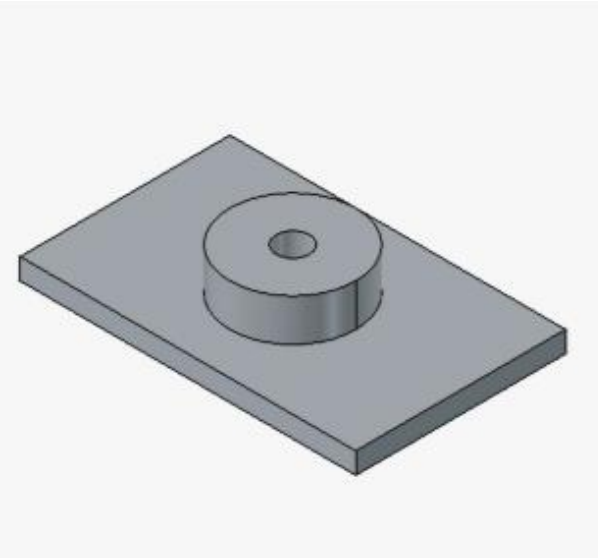
STEP 6 : STEP 3 에서 만들었던 형상을 .JSON에서도 적용이 되는지 확인하기 위해 재시도 (요청값도 동일함)

요청한 자연어값: Create one mechanical mounting part as a single solid in FreeCAD. Create a rectangular base plate with X length 80 mm, Y width 50 mm, and Z thickness 5 mm at $X=0, Y=0, Z=0$. Add one solid cylindrical boss with diameter 30 mm and length 10 mm. Start the boss at $X=40, Y=25, Z=5$ and point it along positive Z. Fuse the boss with the base plate. Create one cylindrical through hole with diameter 8 mm and length 15 mm. Start the hole at $X=40, Y=25, Z=0$ and point it along positive Z. Subtract the hole from the fused part.

[STEP 3]



[STEP 6] ⇒ 똑같은 위치에서 스크린샷 찍었는데, 아예 동일함



⇒ 성공적으로 생성

■ 세번 째 문제 상황: 바꾼 모델이 기대 결과값을 제대로 파악 못함

⇒ 프롬프트 문장을 계속 추가하는 것은 좋은 해결법이 아님. 현재 모델은 코드 생성용 모델이고, 복잡한 자연어 설계 계획 분류에서 한계를 보이고 있다.

새로운 방법: 모델 역할 분리

Qwen3 8B

자연어 → JSON 설계 계획

compile_plan.ps1

JSON → 정확한 Python 코드

FreeCAD

Python → CAD 파일

Qwen2.5-Coder 7B 는 코드 생성용으로 선택했지만, 현재 JSON 방식에서는 LLM이 Python을 작성하지 않습니다. 따라서 자연어 이해와 계획에 더 적합한 범용 모델 (Qwen3 8B) 을 비교해볼 차례

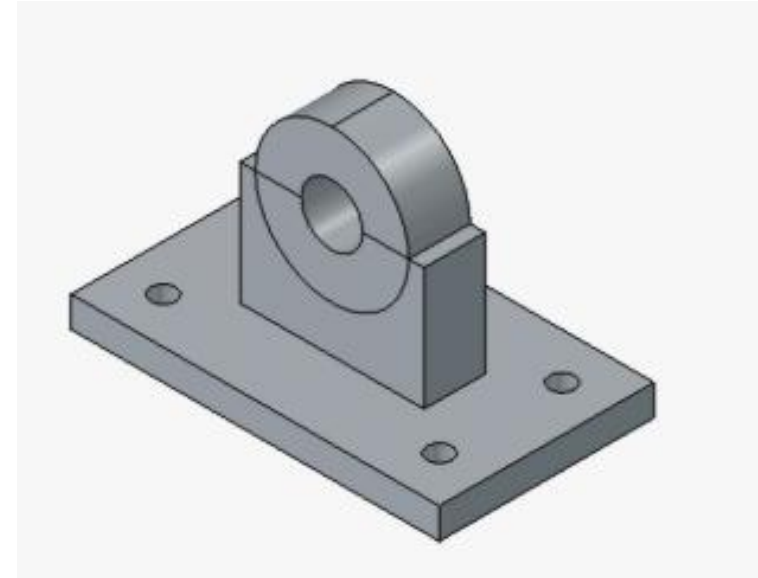
STEP 7 : 베어링 지지대

요청한 자연어: Create one bearing support as a single solid in FreeCAD. Create a rectangular base plate with X length 120 mm, Y width 70 mm, and Z thickness 10 mm at X=0,Y=0,Z=0. Create a rectangular center pedestal with X length 60 mm, Y width 20 mm, and Z height 40 mm at X=30,Y=25,Z=10. Add one solid cylindrical bearing housing with diameter 50 mm and length 20 mm. Start the housing at X=60,Y=25,Z=50 and point it along positive Y. Fuse the base plate, center pedestal, and cylindrical housing. Create four cylindrical mounting holes through the base plate. Each mounting hole has diameter 8 mm, length 10 mm, and points along positive Z. Start the holes at X=15,Y=15,Z=0; X=105,Y=15,Z=0; X=15,Y=55,Z=0; and X=105,Y=55,Z=0. Create one cylindrical shaft hole through the bearing housing with diameter 20 mm and length 20 mm. Start it at X=60,Y=25,Z=50 and point it along positive Y. Subtract all five holes from the fused part.

```
>>> obj = App.ActiveDocument.getObject("Result")
>>> print(obj.Shape.BoundingBox.XLength, obj.Shape.BoundingBox.YLength, obj.Shape.BoundingBox.ZLength)
120.0 70.0 75.0
>>> print("solids=", len(obj.Shape.Solids), "valid=", obj.Shape.IsValid())
solids= 1 valid= True
>>> print(obj.Shape.Volume)
143341.14947945913
```

```
>>> [(print("radius=", e.Curve.Radius, "center=", e.Curve.Center, "axis=", e.Curve.Axis)) for e in obj.Shape.Edges if hasattr(e.Curve, "Radius")]
radius= 4.0 center= Vector (15.0, 15.0, 10.0) axis= Vector (0.0, 0.0, 1.0)
radius= 4.0 center= Vector (105.0, 15.0, 10.0) axis= Vector (0.0, 0.0, 1.0)
radius= 4.0 center= Vector (15.0, 55.0, 10.0) axis= Vector (0.0, 0.0, 1.0)
radius= 4.0 center= Vector (105.0, 55.0, 10.0) axis= Vector (0.0, 0.0, 1.0)
radius= 4.0 center= Vector (15.0, 15.0, 0.0) axis= Vector (0.0, 0.0, 1.0)
radius= 4.0 center= Vector (105.0, 15.0, 0.0) axis= Vector (0.0, 0.0, 1.0)
radius= 4.0 center= Vector (15.0, 55.0, 0.0) axis= Vector (0.0, 0.0, 1.0)
radius= 4.0 center= Vector (105.0, 55.0, 0.0) axis= Vector (0.0, 0.0, 1.0)
radius= 25.0 center= Vector (60.0, 25.0, 50.0) axis= Vector (0.0, 1.0, 0.0)
radius= 25.0 center= Vector (60.0, 45.0, 50.0) axis= Vector (0.0, 1.0, 0.0)
```

```
radius= 10.0 center= Vector (60.0, 45.0, 50.0) axis= Vector (0.0, 1.0, 0.0)
radius= 25.0 center= Vector (60.0, 45.0, 50.0) axis= Vector (0.0, 1.0, 0.0)
radius= 25.0 center= Vector (60.0, 25.0, 50.0) axis= Vector (0.0, 1.0, 0.0)
radius= 10.0 center= Vector (60.0, 25.0, 50.0) axis= Vector (0.0, 1.0, 0.0)
radius= 10.0 center= Vector (60.0, 25.0, 50.0) axis= Vector (0.0, 1.0, 0.0)
radius= 25.0 center= Vector (60.0, 25.0, 50.0) axis= Vector (0.0, 1.0, 0.0)
radius= 10.0 center= Vector (60.0, 45.0, 50.0) axis= Vector (0.0, 1.0, 0.0)
radius= 10.0 center= Vector (60.0, 45.0, 50.0) axis= Vector (0.0, 1.0, 0.0)
radius= 25.0 center= Vector (60.0, 45.0, 50.0) axis= Vector (0.0, 1.0, 0.0)
[None, None, None, None, None, None, None, None, None, None, None, None, None, None, None, None, None, None, None]
```



⇒ 성공적으로 생성

이번 단계에서 중요한 성과는 단순히 형상을 만든 것뿐 아니라:

자연어 → 단일 feature 목록 → 자동 분류 → 검증된 plan.json
→ 결정론적 Python 생성 → FreeCAD 단일 솔리드

■ 현재까지 어떤 방식으로 실행이 되는가?

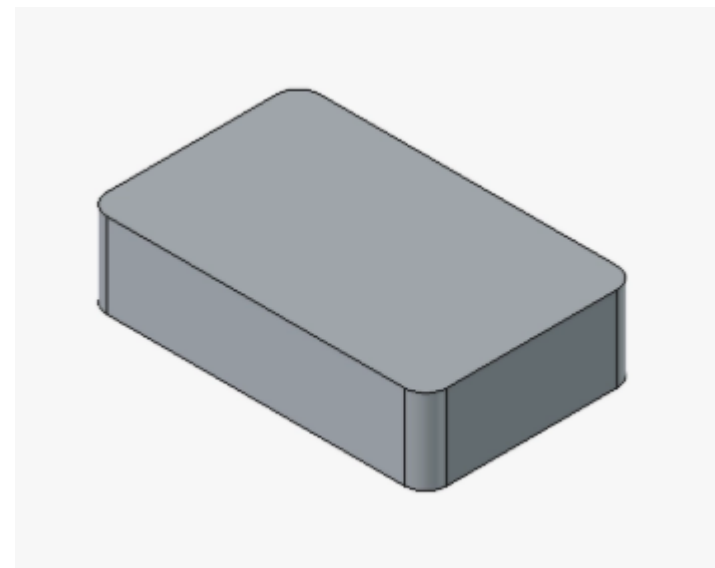
1. 파일에서 터미널 실행
2. Powershell -ExecutionPolicy Bypass -File .\ask_plan.ps1 (코드 입력)
3. 요청 사항 입력
4. 기대 결과값이 프롬프트에 뜨면 정확한지 확인
(만약에 기대 결과값이 다르다면, AI를 통해서 ask_plan.ps1을 열고 \$requirementsPrompt를 수정 하는 방식)
5. (추가적인 확인 과정) 방금 코드 입력을 통해 업데이트 된 plan.JSON 파일도 잘 입력되었는지 확인
6. 다시 터미널에 powershell -ExecutionPolicy Bypass -File .\compile_plan.ps1 코드를 입력해서
7. PLAN_COMPILE_SUCCESS(성공 메시지)와 계산 결과가 잘 나오면 (generated.py에 정상 반영 완료)
8. powershell -ExecutionPolicy Bypass -File .\run_freecad.ps1 코드 입력
9. Free CAD가 켜지면서 형상 확인 (최종)

STEP 8 : 필렛이 추가된 육면체

요청한 자연어값: Create one rectangular solid block as a single solid in FreeCAD. The block has X length 80 mm, Y width 50 mm, and Z height 20 mm at X=0,Y=0,Z=0. Apply a fillet with radius 5 mm to all straight edges parallel to positive Z. Add only the final shape to the document as one Part::Feature named Result.

```
>>> obj = App.ActiveDocument.getObject("Result")
>>> print(obj.Shape.BoundingBox.XLength, obj.Shape.BoundingBox.YLength, obj.Shape.BoundingBox.ZLength)
80.0 50.0 20.0
>>> print("solids=", len(obj.Shape.Solids), "valid=", obj.Shape.isValid())
solids= 1 valid= True
>>> print(obj.Shape.Volume)
79570.7963267949
```

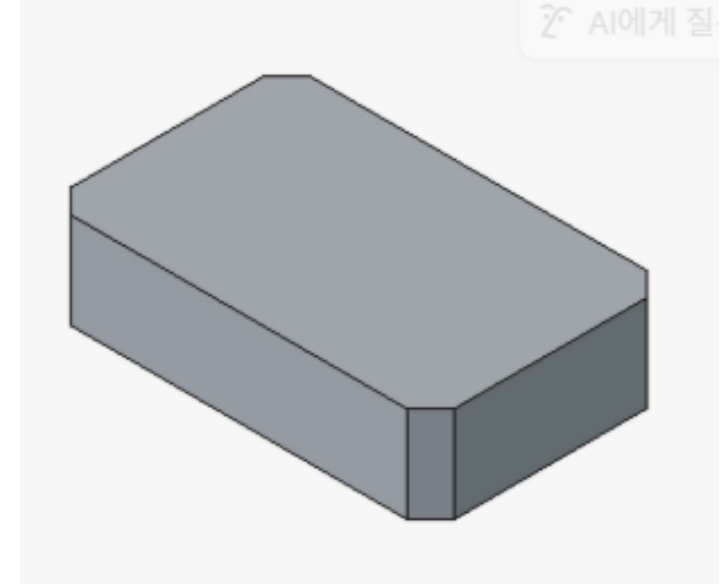
```
>>> [(print("radius=", e.Curve.Radius, "axis=", e.Curve.Axis)) for e in obj.Shape.Edges if hasattr(e.Curve, "Radius")]
radius= 5.0 axis= Vector (-0.0, 0.0, 1.0)
radius= 5.0 axis= Vector (-0.0, -0.0, -1.0)
radius= 5.0 axis= Vector (-0.0, -0.0, -1.0)
radius= 5.0 axis= Vector (-0.0, 0.0, 1.0)
radius= 5.0 axis= Vector (-0.0, 0.0, 1.0)
radius= 5.0 axis= Vector (-0.0, -0.0, -1.0)
radius= 5.0 axis= Vector (-0.0, -0.0, -1.0)
radius= 5.0 axis= Vector (-0.0, 0.0, 1.0)
[None, None, None, None, None, None, None, None]
```



⇒ 성공적으로 생성

STEP 9 : 모서리가 깎인 육면체

요청한 자연어값: Create one rectangular solid block as a single solid in FreeCAD. The block has X length 80 mm, Y width 50 mm, and Z height 20 mm at X=0,Y=0,Z=0. Apply a chamfer with distance 5 mm to all straight edges parallel to positive Z. Do not apply any fillets. Add only the final shape to the document as one Part::Feature named Result.



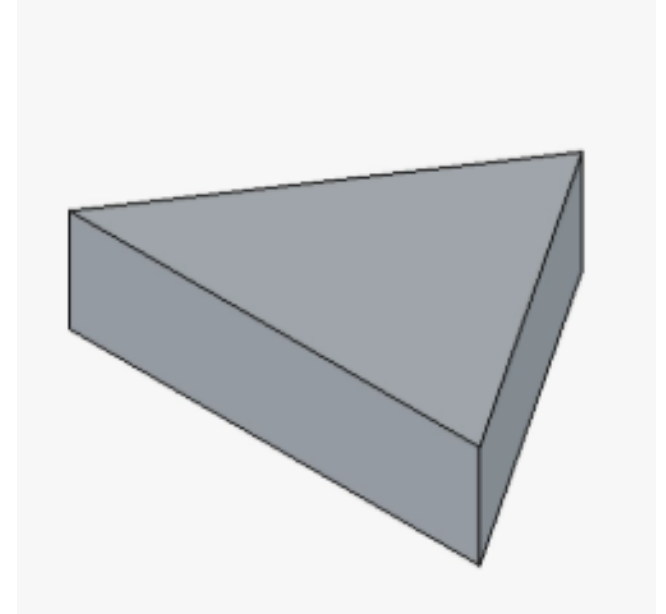
```
>>> obj = App.ActiveDocument.getObject("Result")
>>> print(obj.Shape.BoundingBox.XLength, obj.Shape.BoundingBox.YLength, obj.Shape.BoundingBox.ZLength)
80.0 50.0 20.0
>>> print("solids=", len(obj.Shape.Solids), "valid=", obj.Shape.isValid())
solids= 1 valid= True
>>> print(obj.Shape.Volume)
79000.0
>>> print(sorted(set((round(v.Point.x,3),round(v.Point.y,3),round(v.Point.z,3)) for v in obj.Shape.Vertexes)))
[(0.0, 5.0, 0.0), (0.0, 5.0, 20.0), (0.0, 45.0, 0.0), (0.0, 45.0, 20.0), (5.0, 0.0, 0.0), (5.0, 0.0, 20.0), (5.0, 50.0, 0.0), (5.0, 50.0, 20.0), (75.0, 0.0, 0.0), (75.0, 0.0, 20.0), (75.0, 50.0, 0.0), (75.0, 50.0, 20.0), (80.0, 5.0, 0.0), (80.0, 5.0, 20.0), (80.0, 45.0, 0.0), (80.0, 45.0, 20.0)]
```

⇒ 성공적으로 생성

STEP 10 : 삼각형 프로파일 돌출 시험

요청한 자연어값: Create one triangular profile prism as a single solid in FreeCAD. Use the three absolute profile points [0,0,0], [40,0,0], and [20,30,0]. Create a closed planar face from these points and extrude it using the vector [0,0,10]. Do not create any boxes, cylinders, holes, fillets, or chamfers. Add only the final shape to the document as one Part::Feature named Result.

```
>>> obj = App.ActiveDocument.getObject("Result")
>>> print(obj.Shape.BoundingBox.XLength, obj.Shape.BoundingBox.YLength, obj.Shape.BoundingBox.ZLength)
40.0 30.0 10.0
>>> print(obj.Shape.Volume, len(obj.Shape.Solids), obj.Shape.isValid())
6000.0 1 True
>>>
```



⇒ 성공적으로 생성

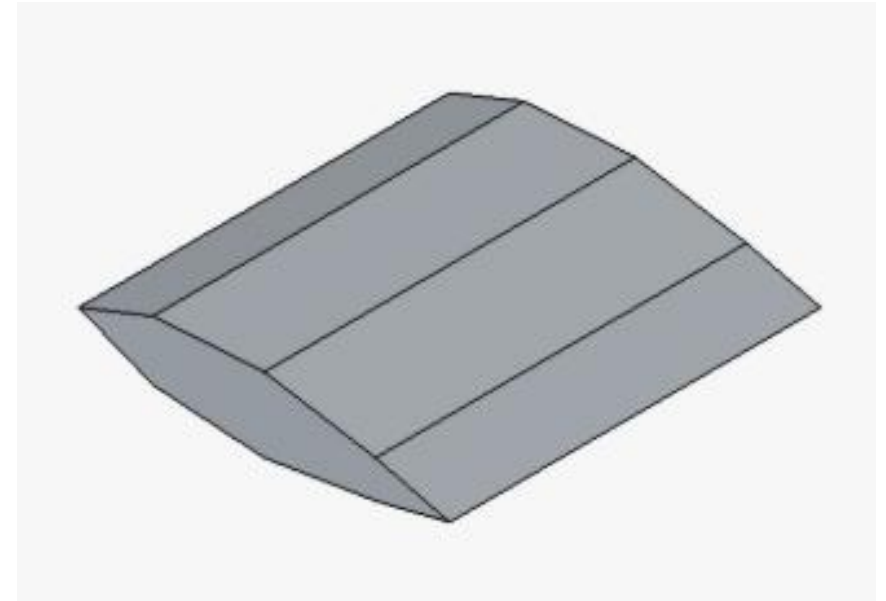
[해당 단계의 의미]:

이제 해당 시스템은 박스·원기둥뿐 아니라 여러 좌표로 정의된 임의의 단면을 3D 솔리드로 만들 수 있다.

STEP 11 : 다점 날개형 프로파일

요청한 자연어값: Create one wing-shaped polygonal profile prism as a single solid in FreeCAD. Use these eight absolute profile points in the XZ plane in this exact order: [100,0,0], [80,0,5], [50,0,10], [20,0,8], [0,0,0], [20,0,-8], [50,0,-10], [80,0,-5]. Create one closed planar face and extrude it using the vector [0,100,0]. Do not create any boxes, cylinders, holes, fillets, or chamfers. Add only the final shape to the document as one Part::Feature named Result.

```
>>> obj = App.ActiveDocument.getObject("Result")
>>> print(obj.Shape.BoundingBox.XLength, obj.Shape.BoundingBox.YLength, obj.Shape.BoundingBox.ZLength)
100.0 100.0 20.0
>>> print(obj.Shape.Volume, len(obj.Shape.Solids), obj.Shape.isValid())
124999.99999999999 1 True
```

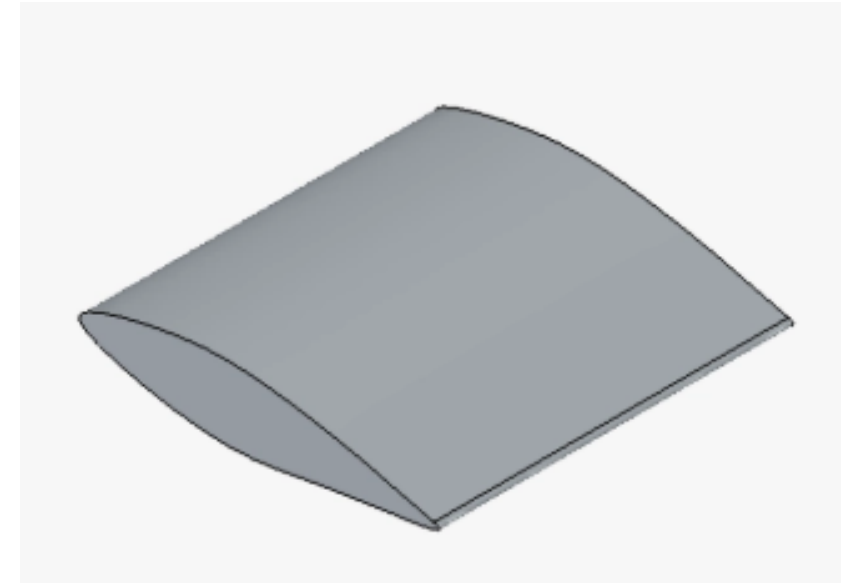


⇒ 성공적으로 생성

STEP 12 : 매끄러운 곡면 프로파일

요청한 자연어값: Create one smooth wing-shaped profile prism as a single solid in FreeCAD. Use curve_type periodic_bspline. Use these eight absolute interpolation points in the XZ plane in this exact order: [100,0,0], [80,0,5], [50,0,10], [20,0,8], [0,0,0], [20,0,-8], [50,0,-10], [80,0,-5]. Create one smooth closed periodic B-spline profile and extrude it using the vector [0,100,0]. Do not create any boxes, cylinders, holes, fillets, or chamfers. Add only the final shape to the document as one Part::Feature named Result.

```
>>> obj = App.ActiveDocument.getObject("Result")
>>> print(obj.Shape.BoundingBox.XLength, obj.Shape.BoundingBox.YLength, obj.Shape.BoundingBox.ZLength)
101.57544836230706 100.0 20.830123218608364
>>> print(obj.Shape.Volume, len(obj.Shape.Solids), obj.Shape.isValid())
137723.93955497013 1 True
```



⇒ 완전 성공은 아님. 치수 부분에서 조금의 오차 발생 했지만 다음 STEP에서 해결할 예정.

STEP 13 : 실제 NACA 0012 날개

⇒ 이번부터는 LLM이 에어 포일 좌표를 만들지 않고, LLM은 다음 설계 값만 구조화 한 뒤,
Python이 NACA 공식을 이용해 좌표를 결정론적으로 계산하게 만들

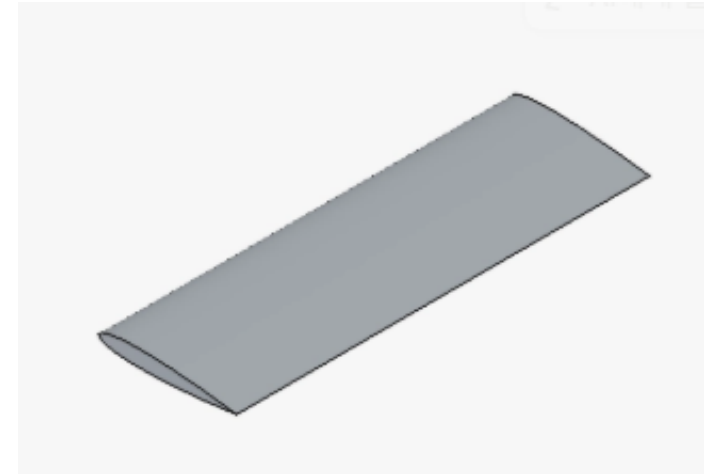
Why?: 이렇게 바꾼 가장 큰 이유는 치수 정확도와 재현성 때문이다.

LLM이 에어포일 좌표를 직접 만들면 다음 문제

- 같은 요청에도 좌표가 매번 조금씩 달라질 수 있음
- 좌표 일부가 누락되거나 순서가 바뀔 수 있음
- NACA 0012라고 말해도 실제 좌표가 공식과 정확히 일치하는지 보장하기 어려움
- 샘플 수가 많아질수록 숫자 복사 오류와 반올림 오차가 증가함
- 형상은 날개처럼 보여도 실제 두께 분포가 틀릴 수 있음

그래서 역할을 분리

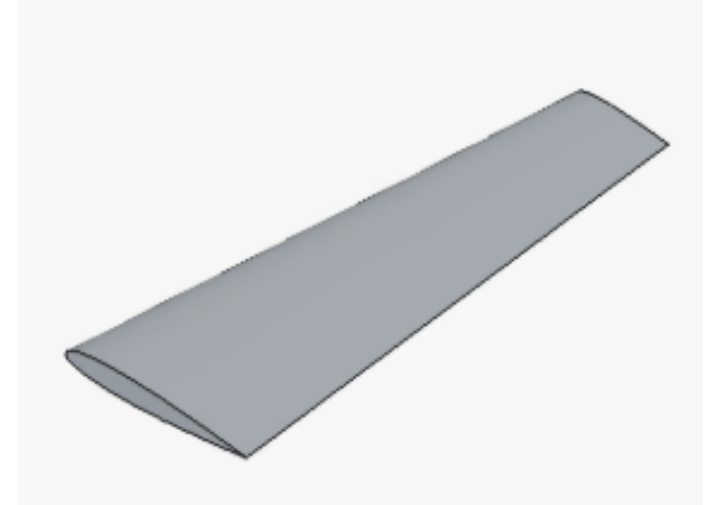
요청한 자연어값: Create one NACA 0012 wing as a single solid in FreeCAD. Use the four-digit designation 0012, chord length 100 mm, span 300 mm, 101 samples per surface, a closed trailing edge, and position [0,0,0]. Generate the airfoil coordinates deterministically from the NACA four-digit equations. Do not create any boxes, cylinders, holes, profile prisms, fillets, or chamfers. Add only the final shape to the document as one Part::Feature named Result.



```
>>> obj = App.ActiveDocument.getObject("Result")
>>> print(obj.Shape.BoundingBox.XLength, obj.Shape.BoundingBox.YLength, obj.Shape.BoundingBox.ZLength)
100.0 300.0 11.994295561397326
>>> print(obj.Shape.Volume, len(obj.Shape.Solids), obj.Shape.isValid())
245111.5571068999 1 True
```

STEP 14 : 루트·팁 단면을 연결한 날개

요청한 자연어값: Create one tapered NACA 0012 loft wing as a single solid in FreeCAD. Use NACA designation 0012, root chord 120 mm, tip chord 60 mm, span 300 mm, sweep offset 40 mm, tip Z offset 20 mm, 101 samples per section, a closed trailing edge, and position [0,0,0]. Generate the root and tip airfoil sections deterministically from the NACA four-digit equations and connect them using a solid loft. Do not create any boxes, cylinders, holes, straight-extruded NACA wings, profile prisms, fillets, or chamfers. Add only the final shape to the document as one Part::Feature named Result.



```
>>> obj = App.ActiveDocument.getObject("Result")
>>> print(obj.Shape.Volume, len(obj.Shape.Solids), obj.Shape.isValid())
205783.12102041332 1 True
>>> bb = obj.Shape.BoundingBox
>>> print(bb.XMin, bb.XMax, bb.YMin, bb.YMax, bb.ZMin, bb.ZMax)
0.0 120.0 0.0 300.00000000000006 -7.108114637002378 23.531188395841383
>>> [(round(f.BoundingBox.XMin,6), round(f.BoundingBox.XMax,6), round(f.BoundingBox.YMin,6), round(f.BoundingBox.YMax,6), round(f.BoundingBox.ZMin,6), round(f.BoundingBox.ZMax,6)) for f in obj.Shape.Faces if f.BoundingBox.YLength < 1e-6]
[(0.0, 120.0, 0.0, 0.0, -7.108115, 7.125803), (40.0, 100.0, 300.0, 300.0, 16.400319, 23.531188)]
>>> [(lambda pts: (round(min(p.x for p in pts),6),round(max(p.x for p in pts),6),round(min(p.y for p in pts),6),round(max(p.y for p in pts),6),round(min(p.z for p in pts),6),round(max(p.z for p in pts),6),round(max(p.z for p in pts)-min(p.z for p in pts),6)))([p for e in f.Edges for p in e.discretize(Number=2001)]) for f in obj.Shape.Faces if f.BoundingBox.YLength < 1e-6]
[(0.0, 120.0, 0.0, 0.0, -7.200853, 7.200853, 14.401706), (40.0, 100.0, 300.0, 300.0, 16.399573, 23.600427, 7.200853)]
```

⇒ 성공적으로 생성.

STEP 15 넘어가기 전 네 번째 문제 상황 (스키마 검증만으로는 의미상 치수 오류를 잡지 못한다는 근거)

스키마 검증은 LLM이 만든 JSON 계획이 우리가 정한 형식과 규칙을 지켰는지 검사하는 과정

검증기는 다음을 확인

- `designation` 이 존재하는가?
- NACA 번호가 정확히 네 자리인가?
- `chord` 와 `span` 이 숫자인가?
- 길이가 0보다 큰가?
- `samples` 가 허용 범위 안에 있는가?
- `position` 에 숫자가 정확히 3개 있는가?
- 허용하지 않은 항목을 LLM이 임의로 추가했는가?
- 같은 `id` 를 중복해서 사용했는가?
- 요청한 형상 개수와 JSON의 개수가 같은가?

⇒ 결국 문제 상황은

스키마 검증은 주로 값이 올바른 형식인가를 검사하는데,
그 값이 현재 사용자의 원래 의도와 같은지는 완전히 판단하지 못한다는 뜻

새로운 방식

사용자 원문 치수

- LLM이 JSON 계획 생성
- 원문에서 숫자를 결정론적으로 재추출
- JSON 값과 비교·교정
- 검증 통과 후에만 컴파일

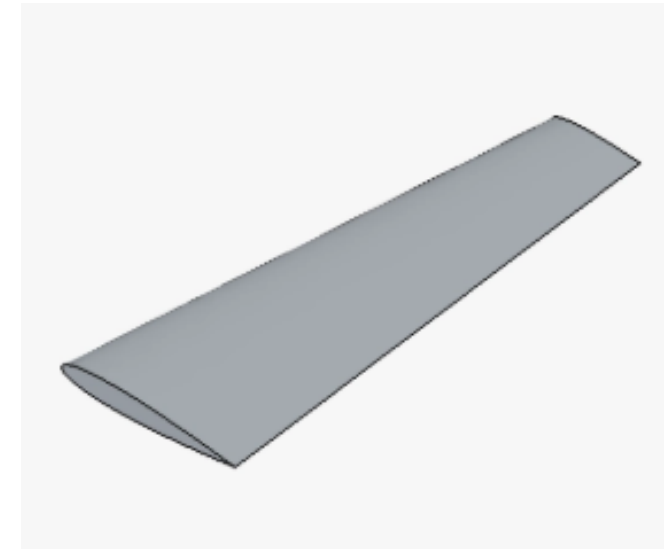
단, 이것이 모든 자유로운 문장을 즉시 완벽하게 이해한다는 뜻은 아니다.
우선 현재 지원하는 NACA loft 필드를 확실하게 고정하고,
이후 box-cylinder-hole에도 같은 방식을 확장할 것

STEP 15 : 날개 팁에 비틀림(twist)이 추가된 로프트 날개

요청한 자연어값: Create one twisted tapered NACA 0012 loft wing as a single solid in FreeCAD. Use NACA designation 0012, root chord 120 mm, tip chord 60 mm, span 300 mm, sweep offset 40 mm, tip Z offset 20 mm, tip twist angle -3 degrees, twist axis fraction 0.25 measured from the tip leading edge, 101 samples per section, a closed trailing edge, and position [0,0,0]. Generate the root and tip airfoil sections deterministically from the NACA four-digit equations, rotate the tip section using the right-hand rule about an axis parallel to positive Y, and connect the sections using a solid loft. Do not create any boxes, cylinders, holes, straight-extruded NACA wings, profile prisms, fillets, or chamfers. Add only the final shape to the document as one Part::Feature named Result.

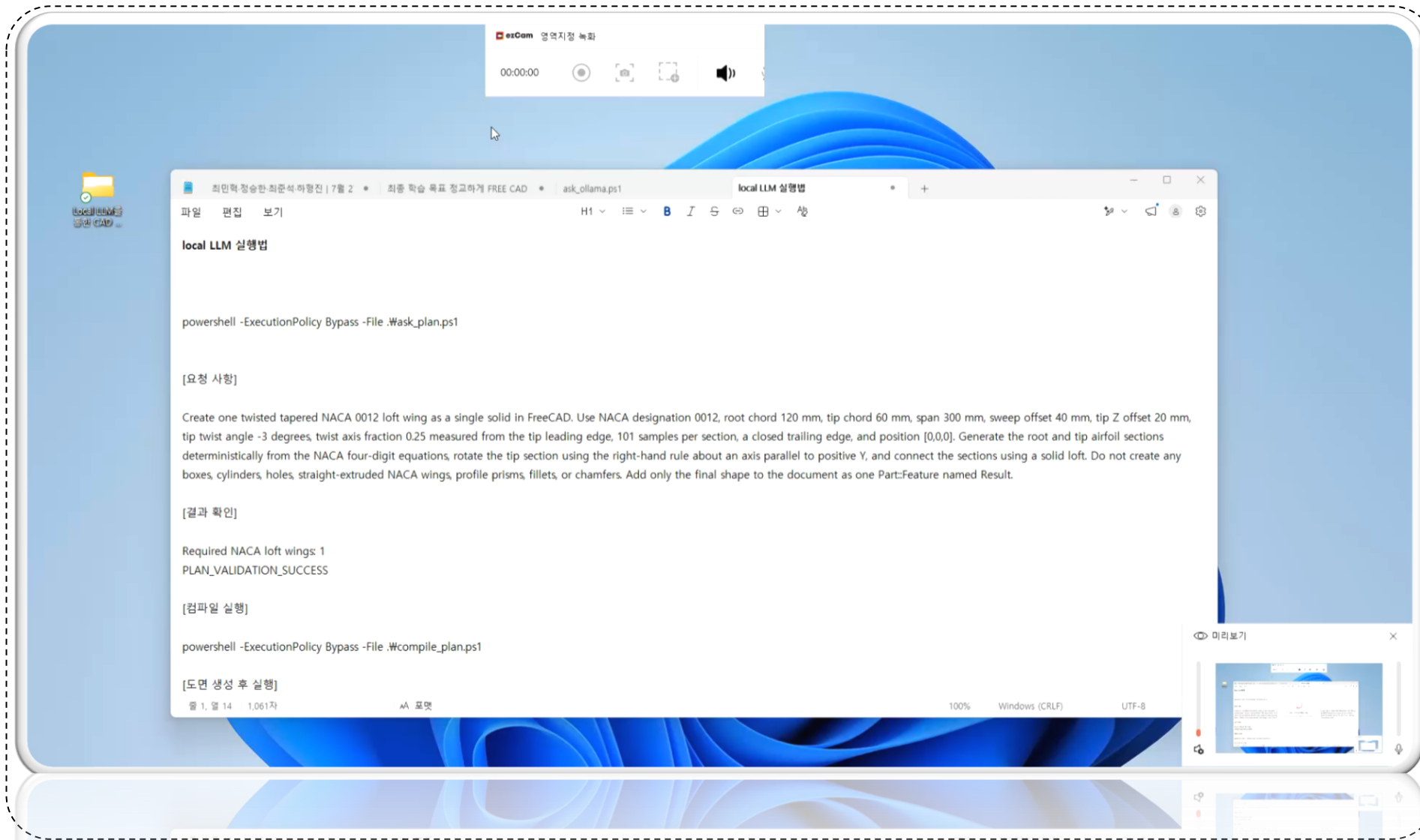
```
>>> obj = App.ActiveDocument.getObject("Result")
>>> print(obj.Shape.Volume, len(obj.Shape.Solids), obj.Shape.isValid())
205702.5698561628 1 True
>>> tip_face = [f for f in obj.Shape.Faces if f.BoundingBox.YLength < 1e-6 and abs(f.BoundingBox.YMin-300) < 1e-6][0]
>>> pts = [v.Point for e in tip_face.Edges for v in e.Vertexes]
>>> le = min(pts, key=lambda p: p.x)
>>> te = max(pts, key=lambda p: p.x)
>>> print(le, te)
Vector (40.02055697868139, 300.0, 19.214960656355853) Vector (99.93832906395583, 300.0, 22.35511803093249)
```

```
>>> import math
>>> print(math.degrees(math.atan2(te.z-le.z, te.x-le.x)))
3.0000000000000008
```



⇒ Meshing이 가능한 정도까지의 CAD 파일 생성 완료

STEP 15의 실행 과정



■ 현재 한계점 / 현재까지 지원 가능한 시스템 / 추후 방향성

✓ 현재 한계점

- 오직 영문만 자연어로 입력
- 영문 입력시 글자 간 띄움 안됨
- AI가 만들어준 형식만 받아들이지 치수를 정확히 말하지 않으면 오류 발생

✓ 추후 방향성

- 피드백 희망

✓ STEP 15까지 지원하는 기능

- 자연어 요청을 로컬 LLM이 JSON 설계값으로 변환
- 박스, 원통, 구멍 생성
- 형상 결합·제거
- 필렛과 모따기
- 다각형 및 B-spline 단면 압출
- NACA 4자리 에어포일 좌표 자동 계산
- 직선 압출 날개 생성
- 테이퍼, 스윙, 높이 변화가 있는 로프트 날개 생성
- 팁 비틀림이 적용된 날개 생성
- JSON 형식·치수 오류 검증 및 일부 자동 교정
- 단일 Solid, 형상 유효성, 부피 및 주요 치수 검증
- FreeCAD, STEP, STP 파일 출력

Any Questions?

이름 최준석
소속 한양대학교 기계공학과
이메일 chw10109@hanyang.ac.kr



*Intelligent Design
for Engineering
Applications Lab*

